Mahathi A J
241801148

Operating System

FIFO Page Replacement Algorithm

Write a C program to implement the FIFO Page Replacement Algorithm. The program should take the number of frames and a reference string (sequence of page requests) as input. For each page in the reference string, determine if it results in a 'Page Hit' or a 'Page Fault'. Display the state of the frames after each request and calculate the total number of page faults and hits.

Input Format

1. The first line of input contains an integer representing the number of pages in the reference string.
2. The second line contains the page reference string as a sequence of integers.
3. The third line contains an integer representing the number of available frames.

Output Format

The total number of Page Faults and Page Hits are displayed

Sample Input 1

7

1 3 0 3 5 6 3

3

Sample Output 1

Enter number of pages in reference string:

Enter the reference string:

Enter number of frames:

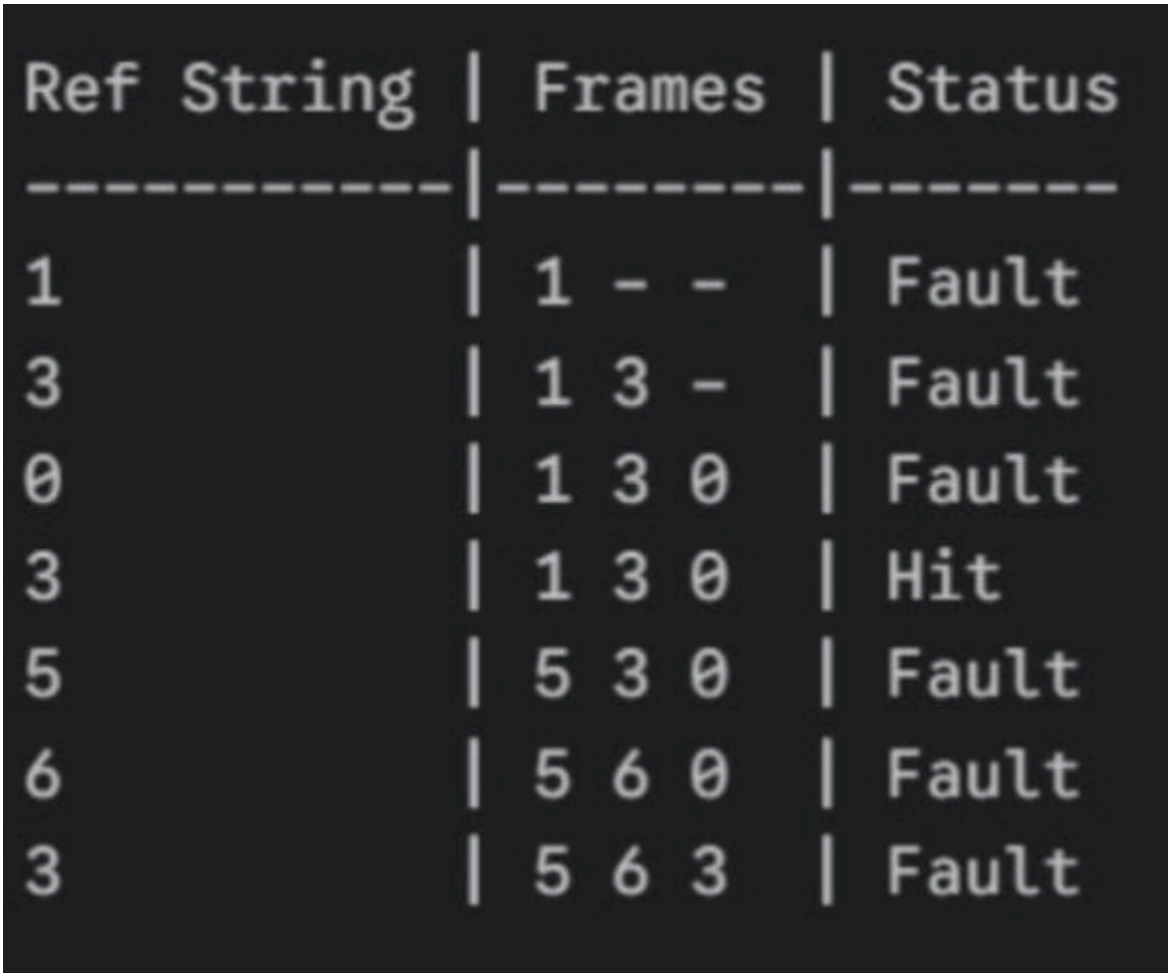
Total Page Faults: 6

Total Page Hits: 1

Mahathi A J
241801148

Operating System

How it works



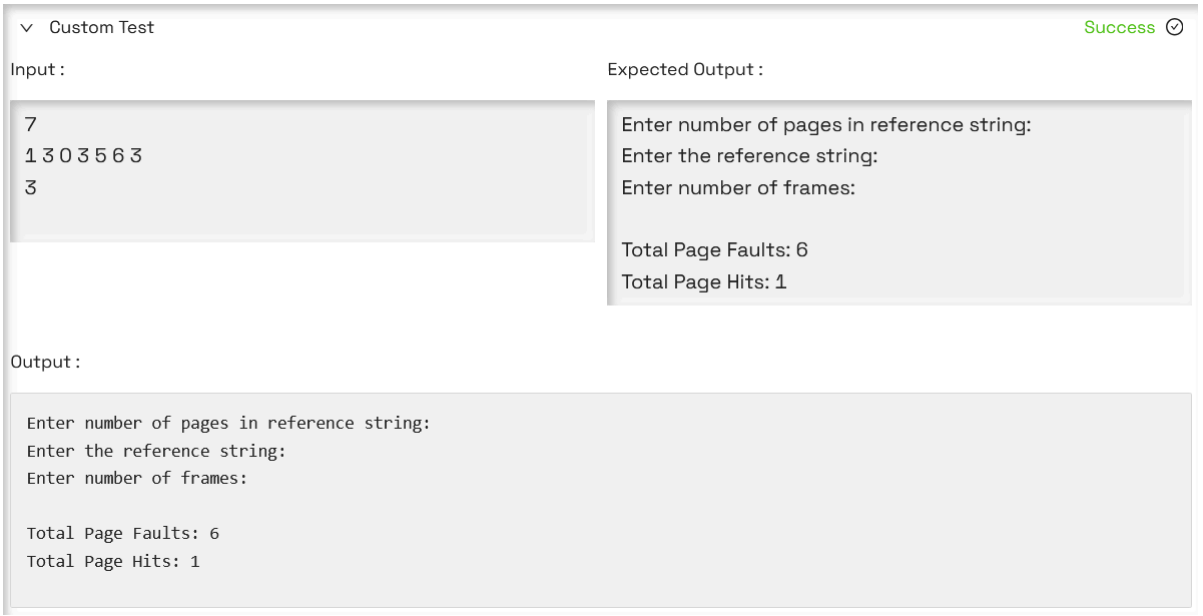
Ref String	Frames	Status
1	1 - -	Fault
3	1 3 -	Fault
0	1 3 0	Fault
3	1 3 0	Hit
5	5 3 0	Fault
6	5 6 0	Fault
3	5 6 3	Fault

Answer:

```
#include <stdio.h>
int main() {
int n, frames;
printf("Enter number of pages in reference string: \n");
scanf("%d", &n);
int ref[n];
printf("Enter the reference string: \n");
for(int i = 0; i < n; i++) {
scanf("%d", &ref[i]);
}
printf("Enter number of frames: \n");
scanf("%d", &frames);
int frame[frames];
for(int i = 0; i < frames; i++) {
```

```
frame[i] = -1;
}
int pageFaults = 0, pageHits = 0;
int index = 0;
for(int i = 0; i < n; i++) {
int found = 0;
for(int j = 0; j < frames; j++) {
if(frame[j] == ref[i]) {
found = 1;
pageHits++;
break;
}
}
if(found == 0) {
frame[index] = ref[i];
index = (index + 1) % frames;
pageFaults++;
}
}
printf("\nTotal Page Faults: %d\n", pageFaults);
printf("Total Page Hits: %d\n", pageHits);
return 0;
}
```

OUTPUT:



Mahathi A J
241801148

Operating System

LRU Page Replacement Algorithm

Write a C program to implement the LRU Page Replacement Algorithm. The program should accept a reference string and the number of frames. For each page request, identify whether it results in a Hit or a Fault. If a Fault occurs and the frames are full, replace the page that was accessed least recently. Display the frame status at each step and calculate the total number of Page Faults and Hits

Input Format

1. The first line of input contains an integer representing the number of pages in the reference string. \
2. The second line contains the page reference string as a sequence of integers.
3. The third line contains an integer representing the number of available frames.

Output Format

The output displays the total number of Page Faults and the total number of Page Hits after processing the entire reference string using the LRU page replacement algorithm.

Sample Input 1

5

1 2 3 4 5

5

Sample Output 1

Enter number of pages in reference string:

Enter the reference string:

Enter number of frames:

Total Page Faults: 5

Total Page Hits: 0

Mahathi A J
241801148

Operating System

Answer:

```
#include <stdio.h>
int main() {
int n, frames;
printf("Enter number of pages in reference string: \n");
scanf("%d", &n);
int ref[n];
printf("Enter the reference string: \n");
for(int i = 0; i < n; i++) {
scanf("%d", &ref[i]);
}
printf("Enter number of frames: \n");
scanf("%d", &frames);
int frame[frames], time[frames];
for(int i = 0; i < frames; i++) {
frame[i] = -1;
time[i] = 0;
}
int pageFaults = 0, pageHits = 0, counter = 0;
for(int i = 0; i < n; i++) {
int found = 0;
for(int j = 0; j < frames; j++) {
if(frame[j] == ref[i]) {
counter++;
time[j] =
counter;
pageHits++;
found = 1;
break;
}
}
if(found == 0) {
int lru = 0;
for(int j = 1; j < frames; j++) {
if(time[j] < time[lru]) {
lru = j;
}
}
counter++;
frame[lru] = ref[i];
time[lru] = counter;
pageFaults++;
}
}
printf("\nTotal Page Faults: %d\n", pageFaults);
printf("Total Page Hits: %d\n", pageHits);
return 0;
}
```

Mahathi A J
241801148

Operating System

OUTPUT:

Custom Test — Success

Input :

```
5
1 2 3 4 5
5
```

Expected Output :

```
Enter number of pages in reference string:
Enter the reference string:
Enter number of frames:

Total Page Faults: 5
Total Page Hits: 0
```

Output :

```
Enter number of pages in reference string:
Enter the reference string:
Enter number of frames:

Total Page Faults: 5
Total Page Hits: 0
```

Name : Mahathi A J
RollNo: 241801148

OPTIMAL

Aim:
To write a c program to implement Optimal page replacement algorithm.

ALGORITHM:
1.Start the process
2.Declare the size
3.Get the number of pages to be inserted
4.Get the value
5.Declare counter and stack
6.Select the least frequently used page by counter value
7.Stack them according the selection.
8.Display the values
9.Stop the process

PROGRAM:

```
#include <stdio.h>

int main() {
   int frames, pages, i, j, k, pos, max, faults = 0;

   printf("Enter number of frames: ");
   scanf("%d", &frames);

   printf("Enter number of pages: ");
   scanf("%d", &pages);

   int ref[pages], frame[frames];

   printf("Enter page reference string:\n");
   for (i = 0; i < pages; i++)
      scanf("%d", &ref[i]);

   for (i = 0; i < frames; i++)
      frame[i] = -1;

   for (i = 0; i < pages; i++) {
```

```
int found = 0;

for (j = 0; j < frames; j++) {
   if (frame[j] == ref[i]) {
      found = 1;
      break;
   }
}

if (found) {
   printf("\nPage %d -> Hit", ref[i]);
} else {
   int empty = -1;
   for (j = 0; j < frames; j++) {
      if (frame[j] == -1) {
         empty = j;
         break;
      }
   }

   if (empty != -1) {
      frame[empty] = ref[i];
   } else {
      max = -1;

      for (j = 0; j < frames; j++) {
         int next = -1;

         for (k = i + 1; k < pages; k++) {
            if (frame[j] == ref[k]) {
               next = k;
               break;
            }
         }

         if (next == -1) { // not used again
            pos = j;
            break;
         }
```

```
                if (next > max) {
                    max = next;
                    pos = j;
                }
            }

            frame[pos] = ref[i];
        }

        faults++;
        printf("\nPage %d -> Fault", ref[i]);
    }
    printf("\tFrames: ");
    for (j = 0; j < frames; j++) {
        if (frame[j] != -1)
            printf("%d ", frame[j]);
        else
            printf("- ");
    }
}

printf("\n\nTotal Page Faults = %d\n", faults);

return 0;
}
```

OUTPUT:

```
Enter number of frames: 3
Enter number of pages: 7
Enter page reference string:
7 0 1 2 0 3 0
Page 7 -> Fault Frames: 7 - -
Page 0 -> Fault Frames: 7 0 -
Page 1 -> Fault Frames: 7 0 1
Page 2 -> Fault Frames: 2 0 1
Page 0 -> Hit Frames: 2 0 1
Page 3 -> Fault Frames: 2 0 3
Page 0 -> Hit Frames: 2 0 3

Total Page Faults = 5
```